

# RegLIC: GPU Realization of Shared Early Exit

Anonymous CVPR submission

Paper ID DRAFT

## 1. GPU Realization of the Shared Early-Exit Decoder

**Question and execution boundary.** The arithmetic saving assigned to a spatial exit map becomes useful only if its masked decoder executes efficiently on a GPU. We therefore implemented an opt-in inference path for the existing e15 DCVC-UF shared-exit decoder. It changes neither checkpoint weights nor the recorded exit maps. The six/twelve-block labels below denote exit depths *within that one e15 network*; none of the independently trained D2/D4/D6 codecs is timed or modified here. The measurement boundary starts with the quantized latent and decoder QP already on the GPU and ends when the padded reconstructed tensor is ready. It includes tile selection, shared-prefix execution, adapters, up-sampling, the output block, and grid-seam repair. It excludes analysis-transform encoding, entropy coding/decoding, bit-stream parsing, acquiring the router map, and host-to-device transfer. A synthesis acceleration is thus distinct from a complete decoder or complete codec acceleration.

**Operator construction.** The dominant DCVC-UF DepthConvBlock contains channel-expanding  $1 \times 1$  projections followed by weighted SiLU and a four-way channel reduction. The stock PyTorch path materializes the expanded  $4C$ -channel tensor and launches separate elementwise and reduction kernels. The Triton path fuses the first projection, activation, and reduction into one kernel that writes only  $C$  channels. It also fuses the second projection with its residual addition, and applies analogous pointwise epilogues to the trunk, early-exit adapters, upsample boundary, and RGB head. A dedicated depthwise kernel supports the actual replicate padding used by e15. The full-frame seam module uses a depthwise-activation pass and one fused pointwise-gate-residual pass. The implementation uses  $\text{tf32} \times 3$  matrix multiplication on an RTX A6000 (SM86); inputs and outputs remain FP32. Unsupported layout, dtype, device, or autograd conditions fall back to the stock operators. The wrapper is installed only after checkpoint loading and `eval()`, and converted inference weights are not saved as training checkpoints.

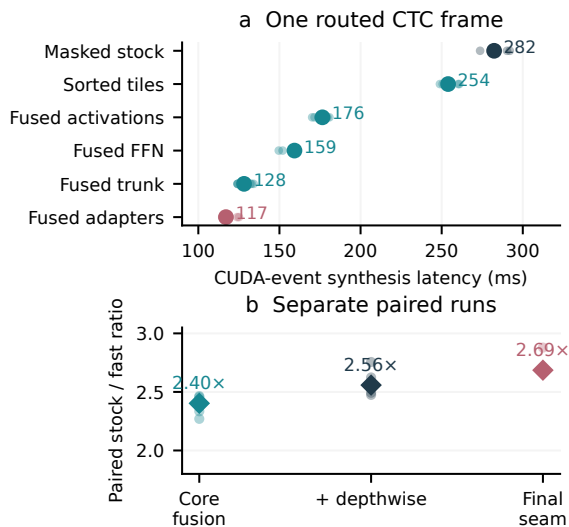


Figure 1. **An implementation gain on one real routed frame.** **a**, Six shuffled, paired CUDA-event trials of consecutive kernel stages. Dots are trials and large marks are medians. **b**, Three separately timed implementations; ratios are paired *within* each run but cannot be compared as an incremental ablation across runs because GPU contention changed. CTC videoSRC05 at QP32,  $1920 \times 1080$  valid pixels padded to  $2048 \times 1280$ , with exit-6/8/10/12 tile counts 13/20/4/3. The GPU was shared with training.

**Paired GPU measurements.** For videoSRC05 at QP32, we replayed its archived source-calibrated router map, containing 40 tiles and four active depths. Each six-trial block used the same latent, QP, weights, and map, with execution order shuffled and TF32 disabled. CUDA-event medians fell from 282.3 ms for the stock masked path to 254.0 ms after depth-sorted execution, 176.5 ms after activation fusion, 159.3 ms after FFN fusion, 128.2 ms after trunk fusion, and 117.0 ms after adapter fusion. The median of six *paired* stock/fast ratios is  $2.40\times$  (individual ratios  $2.27\text{--}2.46\times$ ). This is a measured synthesis gain at one map and QP, not the average speedup of the CTC corpus.

A subsequent depthwise-enabled version gave 287.2 ver-

053 sus 112.1 ms and a median paired ratio of  $2.56\times$  in four  
054 further trials on the same frame. The host-wall medians  
055 were 289.5 and 114.6 ms, respectively. Adding the seam  
056 fusion produced 308.8 versus 113.3 ms in a separate four-  
057 trial run. The latter stock measurement drifted upward under  
058 shared GPU load; its apparent  $2.69\times$  ratio does *not* establish  
059 an incremental gain over the preceding version. A direct  
060 16-pair seam-only ablation within the optimized decoder  
061 measured 116.8 versus 114.8 ms ( $1.035\times$  paired median),  
062 a small contribution consistent with the remaining whole-  
063 frame work.

064 **Numerical fidelity and memory.** The final path was  
065 checked against stock masked execution on five real  
066 CTC frame/QP/map cases: videoSRC05 at QP 0/32/63,  
067 videoSRC01 at QP32, and videoSRC10 at QP32. The largest  
068 absolute reconstructed-sample difference was  $6.26 \times 10^{-7}$ ;  
069 the largest absolute change in the upstream YUV 6:1:1 PSNR  
070 was  $2.09 \times 10^{-7}$  dB. Tile sorting alone was bit-exact; fused  
071 dot products are not, owing to FP32 operation order. For the  
072 same videoSRC05 map, the incremental active-tensor alloca-  
073 tor peak fell from 1,069.5 to 692.1 MB, a 35.3% reduction.  
074 This excludes model weights, reserved CUDA memory, and  
075 allocations held by other processes.

076 **Limits and next benchmark.** These measurements estab-  
077 lish that the early-exit architecture can execute substantially  
078 faster in one GPU synthesis setting, but the A6000 was not  
079 exclusive. Other processes can change clock state, cache  
080 behaviour, and interference between paired blocks. The  
081 53-sequence  $\times$  five-QP benchmark is prepared with CUDA-  
082 event and wall-clock timings, checkpoint and source hashes,  
083 and a refusal to run by default when another compute pro-  
084 cess occupies the selected GPU. It has not been executed.  
085 The final publication claim should come from that isolated  
086 benchmark, with sequence-cluster uncertainty and separate  
087 QP slices, followed by an end-to-end bitstream decode that  
088 charges entropy recovery, map signalling, and any online  
089 router cost. The paper bundle contains the six raw GPU  
090 records, a hash manifest, and a CPU audit script reproducing  
091 every number in Fig. 1.

092 **Why one map is insufficient.** The archived 53-sequence  
093 map inventory already shows a strong QP-dependent work-  
094 load shift: exit 6 receives 79% of selected tiles at QP0 but  
095 37% at QP63; the mean retained trunk depth rises from 6.57  
096 to 8.24 blocks. Different tile occupancies change the number  
097 and shape of pointwise matmuls, gather/scatter traffic, and  
098 shared-prefix reuse. Even at the same pixel count, one dense  
099 depth and four small occupancy groups can have very differ-  
100 ent launch and memory behaviour. The chosen QP32 frame  
101 has 13/20/4/3 tiles at exits 6/8/10/12 and therefore probes  
102 mixed execution, but it is not a representative-weighted CTC

mean by construction. Two small sequences lack archived  
maps at QP63; the prepared cohort runner marks their dense  
fallback explicitly rather than silently changing the denom-  
inator. The paper should report router-map and fallback  
groups separately.

**Rejected shortcuts.** Several tempting local optimizations  
did not survive the scope check. Pretransposing weights  
improved a large common-map timing by only about 0.7%,  
with an additional packing cost. A separate pixel-shuffle  
kernel saved only 0.024 ms at the  $2\times$  stage and had no mea-  
surable gain at the  $8\times$  output. Smaller pointwise tiles low-  
ered register pressure, but did not give stable whole-frame  
wins; the chosen FFN and pointwise kernels use approxi-  
mately 243 and 251 registers per thread, respectively, without  
spills on SM86. Single-pass TF32 shortened isolated 40-tile  
pointwise work by about 17–18%, yet increased raw out-  
put error to roughly  $8 \times 10^{-6}$  and yielded little gain on the  
large common body. We retained  $\text{tf32}\times 3$  pending a full  
decoder-quality sweep. None of these microbenchmarks was  
promoted into the paper’s speedup number.

**Deployment accounting.** The full codec also performs  
entropy recovery and may need to transmit a mode map. An  
older QP32 decomposition gave separate medians of about  
194.5 ms for entropy and latent preparation and 315.0 ms  
for routed synthesis. Applying the one-frame  $2.56\times$  syn-  
thesis ratio to those older component medians would give  
 $(194.5 + 315.0)/(194.5 + 315.0/2.56) = 1.60$  as a hypo-  
thetical total decoder factor. Since those components were  
recorded at a different time and the kernel ratio comes from  
one map, even this Amdahl estimate is not a measured out-  
come. Future native timings must hold map acquisition and  
bitstream representation fixed, report wall time alongside  
CUDA events, and charge the deployment policy for its  
actual control overhead. This distinguishes a faster imple-  
mentation of shared early exit from a new rate–distortion  
operating point.